Where Model-Based Agents Fail on ARC-AGI-3: A Stage-wise Diagnostic Audit

Thomas Serval — NEOMA Business School; Baracoda AI Labs

Abstract

Why do model-based agents fail on unfamiliar interactive environments? We present a stage-wise diagnostic audit for ARC-AGI-3 that locates failures across perception and recognition, transition modelling, goal inference and search, using oracle comparisons on a common population (the first three levels of 20 public games) and five held-out games. Two limitations separate. A symbolic engine that learns game dynamics from its own actions wins 12 of 60 public early levels and none of the first three levels of any held-out game; given source-derived exact rules, our strongest bounded search wins 27 of 60, against 39 for a local LLM agent with no game-specific knowledge (game-level bootstrap of the difference, 95 % [1, 23.5] levels). The engine's logs locate the immediate bottleneck before model learning: in its 19 held-out openings inside our hybrid, 18 stopped at perception or structure recognition and one ended in defeat; on 104 procedurally generated games outside ARC, an earlier version wins 5. A declarative language with declared read/write sets replays all 4,127 observed transitions of the public games and 177 PuzzleScript games — evidence of expressiveness, not of rule discovery — and makes sufficiency, independence and distinguishability checkable; a chain model shows why downstream improvements added no solved level and when a symbolic gate pays. Our linked submission, ROP5, gates the engine inside the LLM agent: the engine alone finishes the public game G50T, but the hybrid scores below the LLM agent on Kaggle's hidden games (22.55 vs 27.34 and 29.89) and on held-out first levels (11.5 vs 15 of 15). The audit procedure, code and data are released under MIT-0.

1. Introduction

Two families of systems compete on ARC-AGI-3: frontier LLMs that write executable world models in Python and check them by replay, and local LLM agents that play through a REPL inside the Kaggle notebook. A third, older route — learn a theory of the game in a formal language, then plan in it — promises sample efficiency and verifiability. We built engines of that kind, gated one inside an LLM agent, and submitted the hybrid. This paper turns that attempt into a method: a stage-wise audit that tells, with measurements rather than arguments, *where* a model-based agent breaks — perception and recognition, transition modelling, goal inference or search — and which repairs are supported by evidence. Its main empirical finding is that an agent can have an expressive language, sound checks and a working planner and still fail entirely because it does not build the right representation of a new environment.

Contributions.

1. **A reusable stage-wise audit (§4–5).** Oracle comparisons on one population bound what each stage can explain, and a seven-step procedure — record every stage, find a local counter-example, locate the first transformation that loses information, replace it alone with a controlled reference, replay downstream with an off-switch control, classify the evidence — turns a lost level into a demonstrated defect, an intervention-supported cause or an open hypothesis. Any staged agent with logged stages can apply it.
2. **Two limitations separated (§4).** The symbolic pipeline does not carry over to new games (12/60 public early levels, 0 of the first three levels of five held-out games, failing before any hypothesis; 5 of 104 synthetic games), and bounded search remains incomplete with source-derived exact rules (27/60, below the LLM agent's 39/60). These are diagnostic comparisons, not controlled ablations.
3. **Checkable properties and a chain model (§3).** Using the read/write sets that our game language declares, we make sufficiency, independence and distinguishability checkable game by game, each with a short proof and a measured check; two elementary propositions on the chain as a whole help read why our downstream improvements added no level, and state when a gate pays.
4. **A loss map, an ontology audit and an audited hybrid (§5–7).** Nine blocked public games traced stage by stage show heterogeneous first failures; the source-derived programs let us test core-knowledge regularities and measure rule complexity; ROP5 is documented with its negative results, and code, data and preregistrations are public.

We joined the competition late: work started on 30 September 2026 and our first Kaggle submission was on 18 October, so everything reported here was built and measured in about five weeks. The engine calls no LLM; the

agent around it does. The code was largely written by LLM coding agents under human direction.

2. Prior work

Tycho (Lehmann, Aioanei and Vahdati, 2026) builds programmatic world models and reaches a mean of 88.49 RHAE on the 25 public games (100 with the strongest models), noting that matching transitions does not identify the objective. OPINE-World (Courtis, Li and Sanner, 2026) explores by ontology errors first and raises Claude Opus 4.8 from 1.5 % to 78.40 % RHAE; Rodionov (2026a, 2026b) studies executable world models written by coding agents. Kepler (Wu, 2026) reaches 100 RHAE with Claude Opus 5 at about 778 USD, reports that animation frames carry task-relevant information missing from settled grids — the kind of loss our sufficiency check detects — and argues that public-set scores discriminate poorly. Wu, Passenger and Song (2026) find that a coding agent's knowledge tied to the current level transfers poorly (33 of 630 script references cross a task boundary). Tufa Labs' The Duck (Qwen 3.6 27B, local), winner of Milestone 1, plays through a REPL and reports that hand-crafted tools hurt (ARC Prize, 2026). EMPA (Tsividis et al., 2021), the Apperception Engine (Evans et al., 2019) and inductive general game playing (Cropper, Evans and Law, 2019) learn theories from play; ASP for game playing (Thielscher, 2009), unsolvability certificates (Eriksson et al., 2017) and partial-order reduction (Wehrle and Helmert, 2012) supply formal machinery. Lehmann and Vahdati (2026) find that self-improved harnesses gain 8.42 RHAE on development games but 2.80 [−1.44, 7.35] held-out; our engine shows that pattern in extreme form. Three recent results bear on ours. DiG-bench (Battleday et al., 2026) finds that giving Gemini 3.1 Pro the rules of 70 text games raises its wins from 18 to 69: for LLM agents, rule discovery is the bottleneck. Our symbolic planners, given exact rules, stay below an LLM agent that has none, which points instead to the priors that guide search. Banwait (2026) argues that perception, not planning, governs ARC-AGI-3, consistent with where all our held-out openings stopped. Liew (2026) shows that the public games can be reached by trivial heuristics, a caution for any public-set result, including our G50T. To our knowledge no prior ARC-AGI-3 work separates the pipeline's stages with an oracle ladder.

3. Approach: systems and theory

One language, two worlds of games. All our engines share a GDL-style declarative language (Love et al., 2008) with three layers: a protocol (state, legal moves, next state, terminal, goal) with five universal constraints; 17 mechanism contracts in the spirit of VGDL (Schaul, 2013), each declaring roles, parameters, read and write sets and the move that tests it; and the move budget as an axiom (exhaustion is a defeat). One exact program per game reproduces **4,127 of 4,127** observed transitions of the 20 public ARC-AGI-3 games (a no-rule baseline: 420). One interpreter written in the language, taking each game's rules as data, replays **177 of 177** human-made PuzzleScript games — **38,924 of 38,924** moves and 852 of 852 level ends against the reference engine (no-rule baseline: 23 % of moves, 0 ends); the corpus is the deterministic subset that engine accepts, and two games are exact only up to a 30-minute cap. The exact programs were written by reading the games' sources: they show what the language can express, not what an agent can learn, and serve only as oracles.

Online engine. Written in Python, it perceives cells and objects in the current frame, learns a transition model (object pushes, player motion) from the frames that follow its own actions, with candidate mechanisms proposed by an inventor module, and plans over that model; no ASP solver runs at play time. A separate **research inducer** fits declarative theories to recorded play with answer-set programming (ASP); it serves the offline benchmark, the evolutionary search and the loss map.

LLM agent (RO) and linked submission (ROP5). Franzen's public notebook (github.com/da-fr/arc-agi-3-solution, Apache-2.0) runs a local quantised model (albucino/Qwen3.8-Flash-Next-W4A16-FP8PLE on SGLang) through a Python REPL; RO adds deterministic tools after 40 actions in a level (map of observed states, paths, traversal with surprise checks, memory of fatal actions). ROP5 adds a gate: at the start of a game, then at most once per level after 40 actions without progress, the online engine opens in an isolated child process (at most ten at a time; the LLM never waits). The parent alone owns the game and re-checks deadline, terminal state, offered actions, level, observation number and frame hash before each engine action; budgets are bounded (150 actions per level; 90 s wait and 300 s wall per opening; stop at the fourth repetition of an action from the same frame; no reset after a game-over); at the end of every opening the LLM receives a summary of the engine's actions. Thresholds were set on public development games. The engine runs on idle CPU cores (the 20 public games in 74 s of CPU) while the LLM uses the GPU (48 minutes on an RTX PRO 6000 for 20 games). The project used under 400 EUR of rented GPU (about 360 EUR documented), no paid LLM API and no training.

Component	Data it sees	What it computes	What it produces
Online engine (in ROP5)	frames after its own actions	perception, transition model, plan	actions; logged stop reason
Research inducer (offline)	recorded play	ASP theories fitted to transitions	theories, predictions, loss map
Language + exact programs (oracle)	game sources, read by the author	replay, read/write sets, planning	exact replays, independence proofs, planner results
Sufficiency detector (offline)	recorded traces	collisions of represented states	counter-examples with coverage

Only the online engine plays inside the submission; proofs, theories and certificates come from the offline tools.

Properties. Write *R(a)* and *W(a)* for the fluents read and written by the contracts an action *a* triggers; by inertia, *a* changes nothing outside *W(a)*, and which contracts it triggers, and what they write, depend only on the state restricted to *R(a)*. A representation φ maps frames to represented states.

Proposition 1 (sufficiency). If frames $x \neq x'$ have $\varphi(x) = \varphi(x')$ and the same action yields $\varphi(x_+) \neq \varphi(x'_+)$, every predictor $f(\varphi(x), a)$ errs on one of the two transitions, whatever its language, data or budget — it receives identical inputs and returns one output. □ Such a prediction error cannot be removed downstream, only by a richer φ, a longer history or more observation. An observed collision refutes sufficiency; no collision does not certify it, so coverage of revisited state–action pairs must be reported. *Check.* On KA59 three such pairs exist, the frames differing by 18 game pixels the sampler never reads; finer sampling when the detector fires raises visible predictions from 5 to 8 of 13 on reused data, loses one non-visible prediction, and leaves nine other games unchanged.

Proposition 2 (independence). If $W(a) \cap R(b) = W(b) \cap R(a) = W(a) \cap W(b) = \varnothing$, *a* and *b* commute in every state — the Bernstein condition behind partial-order reduction — and the result extends to the shared move budget while its threshold is not crossed (Appendix A). *k* pairwise independent actions reach one state through up to *k!* orders; a transposition table merges the states but still generates the paths, so pruning independent orders can save search. *Check.* On 311 action pairs of the exact programs, the 24 predicted independent all commute (0 false); the 77 commuting pairs the language misses are state-dependent.

Proposition 3 (distinguishability). Theories consistent with all observations can be told apart only by an action, or sequence, in a reachable state where they predict different observations; if they differ in one contract, every separating sequence contains an action that triggers that contract in at least one of the theories. □ If the goal requires identifying the theory and outcomes are binary, at least $\lceil \log_2 |H| \rceil$ experiments are needed in the worst case, and more when experiments split the hypotheses unevenly, though winning does not always require full identification. *Check.* On G50T level 6 a latch and a plate-triggered mover explain every observed frame; the language returns the single separating move (leaving the plate).

Proposition 4 (a level is won only through the whole chain). Let a level be won if and only if stages 1…K (recognition, model, goal, search) all succeed, and let p_k be the probability that stage k succeeds given that stages 1…k−1 did. Then P(win) = p_1 p_2 … p_K, by the chain rule, with no independence assumption. If an intervention raises p_j by Δ and leaves the other conditional probabilities unchanged, P(win) rises by $\Delta \cdot \prod_{i \neq j} p_i$. □ Consequences: when any stage, upstream or downstream, almost always fails, an improvement of another stage that leaves the other conditionals unchanged adds at most $\min_{i \neq j} p_i$ to P(win); and with p_1 = 0 (no game recognised) the chain wins nothing whatever its later stages. The proposition is elementary and offers a post-hoc reading, not a prediction: the KA59 and SK48 repairs improved prediction counts without a measured rise in any stage's success probability, so a gain in levels was not to be expected; with recognition at 0 of 19 openings (§4) the engine could win nothing during its openings. The faster-search and inducer nulls are not explained by it: neither intervention raised its own stage.

Proposition 5 (when a gate pays). Let a gate open the engine on a game; with probability r the engine recognises the game and gains g levels in expectation over the LLM alone, and opening costs c_open levels in expectation through overhead or interference. The expected net gain r·g − c_open is positive iff r > c_open/g. Gating on a recognition test that passes with probability s when the engine would recognise the game and f otherwise gives r·s·g − (r·s + (1−r)·f)·c_open, so, provided g > c_open and the test itself is free, a test with low f makes the gate's expected gain non-negative even when r is small. □ Our data sit on both sides of the threshold: on G50T the engine recognised the game in both passages and won all 7 levels, where RO won 0 and 2; on held-out games, recognition succeeded in 0 of 19 openings (Wilson 95 % upper bound 0.17 treating openings as independent; 0.43 counting games), so at the point estimate any positive cost makes the gate a loss, though the bound does not exclude a gain when c_open < 0.17·g. The proposition does not establish that the cn04 and wa30 losses (§4) are gate costs; it says that a gate

triggered by stagnation alone (f = 1) loses whenever r < c_open/g on new games, which is consistent with ROP5's loss on held-out first levels (11.5 vs 15); the Kaggle comparison is too noisy to bear on it.

Remark. Attributing a ladder gap to one stage would require replacing that stage alone by an oracle under a fixed budget; our rungs change several stages, the budget or the search algorithm at once, so their gaps are comparisons, not the $\Delta \cdot \prod$ terms of Proposition 4, and §5 localises losses level by level instead.

These properties are elementary; their value is that the language makes each of them checkable game by game, through explicit contracts and recorded witnesses. Programmatic world models can support comparable checks when instrumented; the contribution here is their joint use to localise failures across a staged agent.

4. Results I — oracle comparisons

Protocol. ARC-AGI-3 has no separate training split; its 25 public games form the public set. We used 20 of them for development, so results on them are in-sample and reported as such; out-of-sample evidence comes from the five held-out public games and Kaggle's hidden set. Population: the first three levels of the 20 public games not held out (60 levels), plus all 144 levels. Held-out: five public games (vc33, tu93, wa30, s5i5, cn04; 39 levels) with disclosed incidental exposure — source-catalogue access before exclusion, information about one game, quarantined accidental runs — and no use in development, attested by the author. Metric: levels won in the real game, plus native ARC scores. Held-out runs were preregistered before they were run (files in the repository, `docs/preregistration/`): the engine on 23 October, run on 26–27 October (one authorised pass after a first attempt interrupted at the start of cn04), the LLM agent on 3 November and the hybrid on 4 November (two passages each, same model and weights as Kaggle).

Rung	Start state	Budget	Runs	Gating
(a) Engine	game reset	150 actions per level	1	sequential
(b) Exact rules + portfolio planners	true start of each level	60 s per level	1	independent
(b′) Exact rules + goal-directed search	true start of each level	1 M nodes, 300 s per level	1	independent
(c) LLM agent (RO)	game reset	48 min for 20 games, 10 streams	2	sequential
(d) Hybrid (ROP5)	game reset	as (c)	2	sequential

Under sequential gating a level is attempted only after the previous one is won: the engine's 0 of 15 held-out first levels means it failed level 1 of each of the five games and never reached levels 2 and 3. Budgets and gating differ, so the ladder is a set of oracle comparisons, not a controlled ablation. Independent level access removes the preceding-level gate, but differences in algorithms and compute prevent a causal decomposition of the score gaps.

Rung	First 3 levels (/60)	All levels (/144)	Held-out first 3 (/15)	Held-out all (/39)
(a) Engine	12	18	0	0
(b) Exact rules + planners, 60 s	18	—	—	—
(b′) Exact rules + goal search, 300 s	27	—	—	—
(c) LLM agent (RO)	39	60	15	24
(d) Hybrid (ROP5)	43.5	68.5	11.5	23
Per-game best of (a) and (c)	—	69.5	—	—

Wilson 95 % intervals, in levels (treating levels as independent, which overstates precision; game-bootstrap intervals for search and agent are [17, 37] and [31.5, 47]): on the 60 development levels, engine 12 [7, 19], planners 18 [12, 26], goal search 27 [20, 35], LLM agent 39 [31, 46]; on the 15 held-out first levels, engine 0 [0, 3] and LLM agent 15 [12, 15].

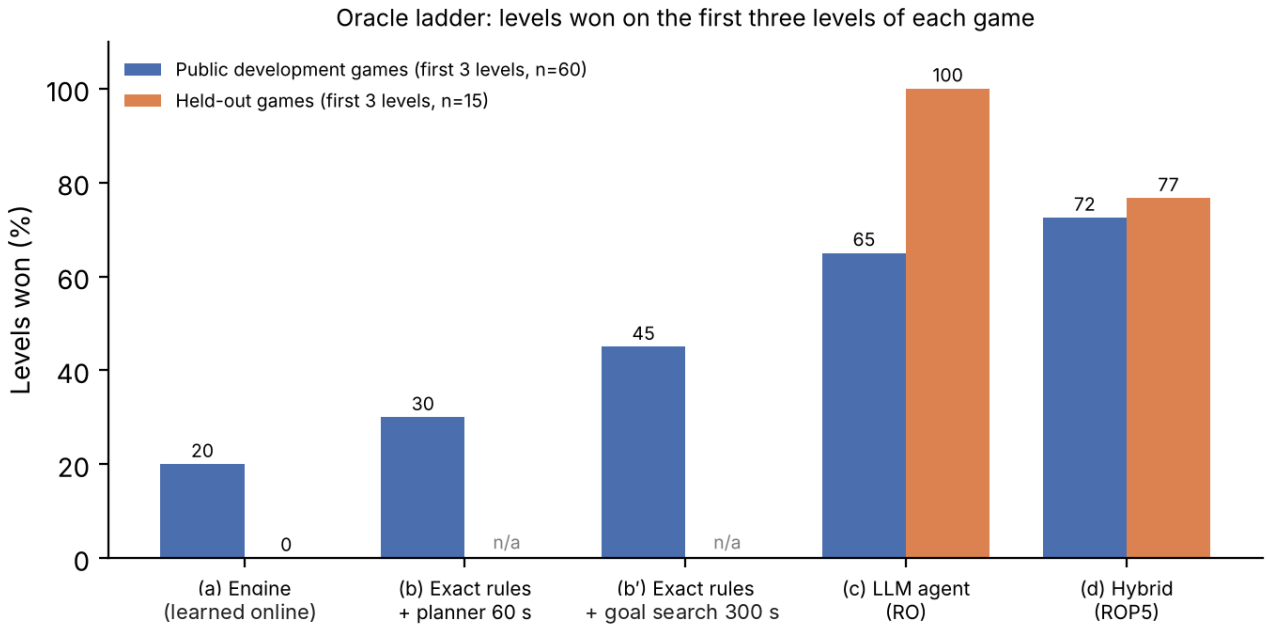


Figure 1. Oracle ladder: levels won on the first three levels of each game, development (n = 60) and held-out (n = 15). Rungs differ in budget and gating.

The symbolic pipeline does not carry over. Six of the engine's 12 early levels are on G50T and KA59, the games it was developed on; on the other 18 development games it wins 6 of 54, so its 0 of 15 held-out is a drop we cannot call significant ($P \approx 0.17$ at that rate), though it never wins a level on a new game. We did not analyse that run's stop reasons separately; the engine's 19 openings inside the hybrid on the same games (two passages) show where it stops: nine ended with *perception not identified* (after one probe per action it could not find which object the actions move), nine with *structure not recognised* (the recogniser, written on development games — its message names G50T — found no point-like goal), one with a game-over. Zero hypotheses were generated, so the transition learner, inventor and planner were never reached. The chain broke at its first stage, before any learning. This is consistent with the representation being binding, but it is a recogniser failure, not a demonstrated violation of Proposition 1: the sufficiency check on these short traces (54 engine actions, at most four per opening) found no collision and had no power.

A larger synthetic check. On 104 procedurally generated games outside ARC (81 keyboard mazes, 23 click tile puzzles; our own generator, used during development, so not held out), an earlier version of the same online chain (25 October; 80 actions and 240 s per game) won 5 first levels (4.8 %, Wilson 95 % [2.1, 10.8]), stopped without an admissible action on 57 and lost 42. All five wins are keyboard games; on the 23 click games the chain made no initial directional probe and won nothing (0 of 23, Wilson upper bound 14 %). The dominant refusals in its logs are "transitions needed" and "no matched mobile object": a bottleneck at perception and recognition similar to the held-out one, on 104 games (about twenty times the five held-out games), with an earlier version of the chain.

Knowing the rules is not enough. With exact rules, our best search wins fewer early levels than the LLM agent (27 vs 39; 23 for the same search on a slower machine). Paired level by level with each RO passage, on the levels won by exactly one of the two the agent wins 16 to 6 (exact two-sided sign test, $p = 0.052$) and 19 to 5 ($p = 0.007$); counting a level for RO only when both passages win it, 15 to 9 ($p = 0.31$). Levels of one game are not independent, so we also resample games: the agent's mean advantage is 12 levels, game-bootstrap 95 % [1, 23.5]; per game, the agent is ahead on 10, the search on 5, tied on 5. The 20 levels the agent wins in at least one passage and the search misses are not longer — median human solution 32.5 actions, against 40 for the levels the search wins; the 13 levels nobody wins are long (median 67). We then tested Proposition 2. The 24 proved pairs never occur among the candidate actions of blocked levels, so pruning them changes nothing. Extending the commutation rule to all clicks of DC22, FT09, SC25 and TN36 — with no counterexample observed in the evaluated runs (identical plans, no level lost), not proved generally safe — cuts generated nodes by 45 % on FT09 and 77 % on TN36 (TN36 level 1 in 0.6 s instead of 24.1 s) but wins no new level at 60 s or 300 s, although the reduced search explores up to four times more states; a compiled simulator 1.66 times faster also wins the same levels. The tested optimisations reduced runtime without increasing solve counts under the budgets evaluated.

The hybrid approaches complementarity where the engine was built, and idles elsewhere. On public games ROP5 obtains 68.5 levels against 69.5 for the per-game best of engine and agent (not an upper bound: a gate can switch within a game), and 36.44 against 30.73 for RO in native score (paired +5.71, game-bootstrap 95 % [−4.02,

+17.81]); G50T and KA59 contribute +7.05 points to that mean while the other 18 games contribute −1.35. In both public ROP5 passages the engine alone finished G50T, all 7 levels in 360 actions without a single LLM token, where RO won 0 and 2 levels; it is a development game, so this shows what the engine does once it recognises a game, not transfer. Counting whole games finished, ROP5 completed 2 and 4 public games per passage (G50T, SB26; FT09, G50T, LP85, SB26) against 2 and 3 for RO (FT09, SB26; FT09, LP85, SB26). On the held-out games ROP5 averaged **45.50** over ten slots (passages 50.50 and 40.50) against 42.98 for RO (41.55 and 44.42). The preregistered prediction (ROP5 ≤ 42.98, the RO score) is refuted in direction, but this comparison does not establish a positive effect (three games up, two down; sign test p = 1.0), and ROP5 completed slightly fewer levels (23 vs 24). ROP5 won a whole held-out game, vc33 (7 of 7 levels), in both passages, which RO never did (6 and 5 of 7). The engine played 54 of 2,657 actions and won no level during its openings, so it gained nothing directly. The per-game pattern is not uniform noise (Appendix B): ROP5 finished vc33 twice, but lost levels on cn04 (1 of 6 in both passages, against 4 of 6 twice for RO) and wa30 (1 and 2 of 9, against 3 twice), so on held-out first levels the hybrid wins 11.5 of 15 against 15 for RO. On cn04 the engine played one and two actions, yet the LLM made fewer tool calls in the same time budget (123 and 124, against 156 and 136 for RO). The gate's overhead — child processes, per-action checks, the summary added to the context — is a plausible cause that we did not isolate. A gate triggered by stagnation alone can thus cost levels on games where the engine contributes nothing. On Kaggle's hidden games ROP5 scored 22.55 against 27.34 and 29.89 for two identical RO submissions — and the public base notebook scored 28.21 and 0.00 on two dates — so we establish no reliable effect of the gate there either.

Other observations. Feeding the LLM agent more context did not show a benefit: a pinned factual summary after 40 actions scored 24.82 on Kaggle, and an executable Python model of the game was withdrawn after no measured gain with a small model; these few interventions do not separate the quality, encoding and use of information. When it wins, the agent is economical: over the levels it completes, human baseline actions divided by its actions is about 1.5.

Searching the inducer did not help in the tested space. In an AlphaEvolve-style loop, LLM coding agents proposed thirteen changes to the research inducer under a fixed judge (exact prediction of visibly changing transitions on recorded play, with identity and empty-theory controls). The best reached 1.018 times the baseline against a preregistered threshold of 3; removing all learned laws dropped the metric from 0.250 to 0.110, so the judge could detect change.

5. Results II — where the chain breaks

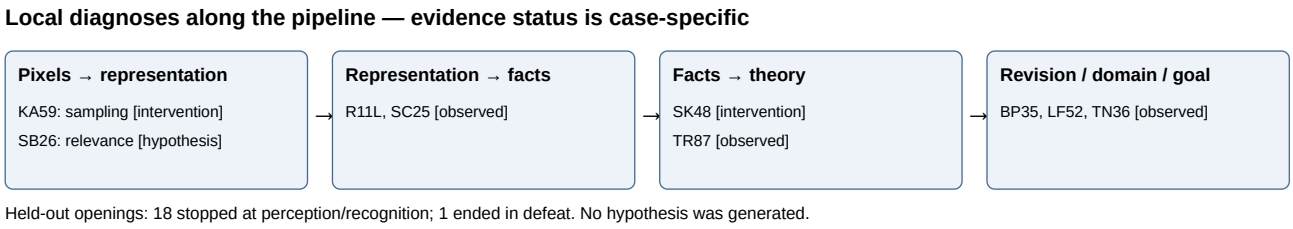


Figure 2. First identified loss (observed, intervention or hypothesis; see table) for the traced public cases. Eighteen held-out openings stopped at perception or recognition; the remaining opening ended in defeat. Status labels distinguish observed defects, intervention evidence and unresolved hypotheses.

Game	First failing stage	Evidence	Status
KA59	Perception (sampling)	same state and action, different successors; 18 pixels never sampled	intervention
SB26	Perception	border pixel lost; relevance unproven	hypothesis
R11L	Facts	destination kept in objects, dropped from ASP facts; guards block 3 of 5 events	observed
SC25	Facts	newly visible support omitted	observed
SK48	Facts → theory	object code compared with pixel colour: 20 false change facts, a 131-cell law on identical images	intervention
TR87	Theory	faithful input; deformed objects counted present without support	observed
BP35	Revision	hypothesis rejected by an observation outside coverage	observed
LF52	Domain	marker attributed to the wrong object; engine refuses to act	observed
TN36	Goal	correct predictions, uninformative goal	observed

Here, *observed* means a defect in a recorded intermediate artefact, *intervention* means a tested local repair with a measured downstream effect, and *hypothesis* means unresolved task relevance. None of these labels alone establishes that repairing the defect would win the level. The status column distinguishes what the logs demonstrate from what an intervention supports: only KA59 and SK48 have a repair that was tested, and SB26 remains a hypothesis; for the other cases the defect is observed but its causal role in the lost level was not tested.

Audit procedure. Inputs are a frozen agent version, staged traces and, when available, a trusted reference for one interface. The output is an auditable case record: stage, witness, controlled replacement, downstream effect, controls and scope. The procedure is: (1) record the inputs and outputs of every stage; (2) look for a local counter-example (two equal inputs with different outcomes, a fact contradicting the frame, a law contradicted by a later observation); (3) find the first transformation where the needed information disappears; (4) where possible, replace only that transformation with a controlled reference (finer sampling, exact facts, exact rules); (5) replay the downstream stages unchanged; (6) measure the effect of the repair and of an off-switch control that must reproduce the unrepaired run exactly; (7) classify the outcome as *defect demonstrated*, *cause supported by intervention* or *unresolved hypothesis*. A missing reference yields an unresolved diagnosis rather than a causal claim. Freeze the intervention before testing it, retain contradictory cases, and distinguish prediction improvement from action selection and level completion. The nine cases were selected for diagnosis, not sampled to estimate failure prevalence.

Intervention and fixed comparison	Observed effect	Limit
KA59: finer sampling when the detector fires; fixed j2 targets, masks and comparator	visible free-rollout predictions 5 → 8 of 13	reused data; one non-visible prediction lost; correlated transitions; no level gain shown
SK48: decode the object code with `couleur_vraie` before comparing it with a pixel colour; semantic key unchanged	20 false change facts removed; the 131-cell error in one event becomes 0; exact prefix predictions 4 → 8 of 30	22 prefix errors remain; on reused suffixes, gains only on 4 stationary transitions under recalibration; no free-rollout improvement
TN36: extended commutation pruning, same tested planner and budget	level 1 planned in 0.6 s instead of 24.1 s; 77 % fewer generated nodes on the reported TN36 aggregate	rule not proved generally safe; no extra level attributable to pruning at 60 s or 300 s

Both local repairs pass an exact off-switch control and were made after the submission; they are development gains, not gameplay gains.

6. Results III — what the ontology reveals about the games

Core knowledge, made executable. ARC is built on core-knowledge priors (Chollet, 2019), and ARC-AGI-3 environments are designed to use only those priors (ARC Prize Foundation, 2026). The language makes them testable:

Core-knowledge prior	Where it lives in the language	Measured status
Objectness, elementary physics	protocol: one anchor per rigid object, inertia	hold in 20/20 public games
Agentness	contracts with roles and read/write sets	17 contracts cover the 20 ARC-AGI-3 games
Goal-directedness, counting	goal predicate; move budget as axiom	budget–defeat in 20/20 public
Geometry, topology	perception of cells, shapes, adjacency	not tested by the language; held-out openings stop at perception (§4) (§4)

Laws that held and a law that failed. Five candidate universal laws — some protocol conventions rather than discovered regularities — hold in every public game: one action per step; never a level end and a defeat on the same step; one anchor per rigid object; inertia; nothing but a reset after a defeat. A sixth, "nothing overlaps a solid", is false in 16 of 20 games and became a parameterised mechanism.

Rule size tracks search difficulty more than LLM difficulty. Exact programs range from 795 (FT09) to 12,062 (BP35) nodes. Program size correlates with the early levels the goal-directed search wins (Spearman $\rho = -0.64$, $n = 20$, $p < 0.01$) but only weakly with the LLM agent's success ($\rho \approx -0.3$, not significant). Size depends on the encoding and its author, so it is not an intrinsic difficulty measure; within this encoding it tracks what symbolic search finds hard; the weaker correlation with the agent is absence of evidence, not evidence of independence.

Describing is not learning. The language covers the tested corpus exactly, yet the online engine learns none of the first held-out levels: the exact programs were written from sources, so this gap is the motivation for the ladder rather than a finding.

7. Discussion

What the engine does contribute. On G50T and KA59 the engine wins 12 of its 18 levels where the LLM agent wins 2.5 on average; inside ROP5 it finishes G50T alone in both reported development passages, using up to 5.6 times fewer actions than the human baseline; it runs on idle CPU cores; every action is checked before it is played, so it never plays an unchecked action, although on cn04 the gate still cost levels (§4); and it records exactly where it stopped. Around it, the language and the offline tools make further objects inspectable — declarative theories, independence proofs, replays against exact programs — though programmatic world models can also be made verifiable. The 19 held-out openings recorded inside the hybrid localise immediate failures to perception or recognition, plus one defeat. This identifies the first obstacle in those openings, not every obstacle that would remain after repairing it.

Priorities for model-based approaches. (1) Search for observable representation contradictions, report revisit coverage, and test recognition beyond the design games before trusting an inducer. (2) Test search priors separately from execution speed: the evaluated speedups added no solve, and we hypothesise, without having isolated it, that exact-rule planners lack knowledge of which sub-goals to pursue — the kind of prior LLM agents bring. (3) Gate symbolic components on evidence that their model fits the current game (no observed collision under measured coverage, an exact replay), not on stagnation alone: by Proposition 5, a recognition test with few false positives keeps a gate's expected gain non-negative on new games, provided the engine's gain exceeds the opening cost. (4) Weight adaptation is a separate hypothesis, motivated by work such as test-time training (Akyürek et al., 2024) and self-adapting models (Zweiger et al., 2025). The present experiments do not establish its benefit.

Limitations. Development data were reused for diagnosis; exact rules were written from game sources; rung budgets differ; held-out results rest on five games and a pod reproduction; Kaggle comparisons comprise a small number of unpaired submissions; two unchanged RO submissions do not estimate the variance of ROP5. The five held-out games have disclosed incidental exposure and should not be described as pristine unseen tasks. Corpus replay establishes agreement on recorded transitions, not correctness on every reachable state.

Availability and provenance. The public linked submission is ARC-AGI-3 Milestone 2 Solution, ROP5 version 9 (native notebook `scriptVersionId` 364167125; score 22.55). The unchanged RO comparison uses `scriptVersionId` 363160941 (27.34 and 29.89). Code and data are public at github.com/Baracoda-ai-labs/arc-agi-3-oracle-ladder, frozen at commit `40135bc`: the language, the exact programs, the online engine and the ROP5 gate, the sufficiency detector, the goal-directed search and the per-run ladder results, with a table mapping each claim of this paper to its files. The notebook link alone does not reproduce the offline audits; the repository does, given the ARC-AGI-3 game files. Exact programs are source-inspected diagnostic oracles, not evidence of autonomous rule

discovery. Author-owned research code is released under MIT-0; upstream components retain their own terms, and unlicensed third-party code is linked rather than redistributed.

8. Conclusion

A symbolic engine that learns and plans in a formal language won levels where it was built — efficiently, cheaply and verifiably — and none where it was not; source-derived rules left the tested planners below the LLM agent in solved-level counts under different budgets. An oracle ladder turns that contrast into a map: the engine breaks at recognition, before reasoning starts; the tested search speedups add no solved level at the evaluated budgets; the context interventions we tested did not establish a benefit. The ladder provides a reproducible way to distinguish upstream recognition failures, model errors and bounded-search limitations. It identifies where interventions should be tested, without establishing that any single intervention will close the performance gap.

Appendix A — Soundness of deduced independences

Model each action a as a step function f_a with read set $R(a)$ and write set $W(a)$; by inertia, f_a changes nothing outside $W(a)$ and what it writes depends only on $R(a)$.

- *Lemma.* If $W(a) \cap R(b) = W(b) \cap R(a) = W(a) \cap W(b) = \varnothing$, then $f_b \circ f_a = f_a \circ f_b$: a fluent in $W(a)$ gets a value computed from $R(a)$, which b never writes, in either order; likewise for $W(b)$; other fluents are untouched.
- *Shared counters.* A fluent written by both is allowed only if each applies $x \leftarrow x \oplus \delta$ with δ independent of x and of the other's writes, and $\oplus$ commutative and associative (the move budget with $\delta = -\text{cost}$; cyclic attributes in $\mathbb{Z}/n$).
- *Budget threshold.* Defeat is read once per step and costs are non-negative: if the reserve after both moves stays above the threshold, both orders are legal and reach the same state; otherwise the pair is not reordered.
- *Autonomous effects* (clock T): apply the lemma to (a, b), (a, T) and (b, T); guards and terminal conditions belong to R.
- *Assumption:* declared sets over-approximate the true ones. A first version with a missing read gave 56 false independences; the corrected language gives 0 of 311.

Appendix B — Held-out games, per passage (native score; levels)

Game	RO P1	RO P2	ROP5 P1	ROP5 P2
vc33	75.00; 6/7	53.26; 5/7	100.00; 7/7	100.00; 7/7
tu93	44.06; 6/9	66.22; 8/9	67.72; 8/9	66.59; 8/9
s5i5	27.78; 4/8	41.67; 5/8	77.78; 7/8	27.78; 4/8
cn04	47.62; 4/6	47.62; 4/6	4.76; 1/6	4.76; 1/6
wa30	13.27; 3/9	13.33; 3/9	2.22; 1/9	3.37; 2/9
Mean	**41.55**	**44.42**	**50.50**	**40.50**

References

- Akyürek, E. et al. (2024). The Surprising Effectiveness of Test-Time Training for Abstract Reasoning. arXiv:2411.07279.
- ARC Prize (2026). ARC Prize 2026: ARC-AGI-3 Milestone Prize #1. arcprize.org.
- ARC Prize Foundation (2026). ARC-AGI-3: A New Challenge for Frontier Agentic Intelligence. arXiv:2603.24621.
- Banwait, H. S. (2026). Transforming Integer Grids to Spatial Intuition: Perception, Not Planning, Governs ARC-AGI-3. arXiv:2611.06793.
- Battleday, R. M., Sandbrink, K. et al. (2026). DiG-bench: Discovery in Games. arXiv:2608.12593.
- Chollet, F. (2019). On the Measure of Intelligence. arXiv:1911.01547.
- Courtis, D., Li, W., Sanner, S. (2026). OPINE-World. arXiv:2607.01531.
- Cropper, A., Evans, R., Law, M. (2019). Inductive general game playing. arXiv:1906.09627.

- Eriksson, S., Röger, G., Helmert, M. (2017). Unsolvability certificates for classical planning. ICAPS.
- Evans, R. et al. (2019). Making sense of sensory input. arXiv:1910.02227.
- Lehmann, J., Aioanei, A., Vahdati, S. (2026). Tycho. arXiv:2607.28287.
- Lehmann, J., Vahdati, S. (2026). Can LLMs Improve Their Own Harness? arXiv:2610.30499.
- Liew, K. H. (2026). Explore Before You Solve: The Speed–Depth Trade-off in Epistemic Agents for ARC-AGI-3. arXiv:2605.25931.
- Love, N. et al. (2008). General Game Playing: Game Description Language Specification. Stanford.
- Rodionov, S. (2026a). Executable World Models for ARC-AGI-3 in the Era of Coding Agents. arXiv:2605.05138.
- Rodionov, S. (2026b). Do Coding Agents Need Executable World Models, Simplification, and Verification to Solve ARC-AGI-3? arXiv:2607.15439.
- Schaul, T. (2013). A video game description language for model-based or interactive learning. IEEE CIG.
- Thielscher, M. (2009). Answer set programming for single-player games in general game playing. ICLP.
- Tsividis, P. et al. (2021). Human-level reinforcement learning through theory-based modeling, exploration, and planning. arXiv:2107.12544.
- Wehrle, M., Helmert, M. (2012). About partial order reduction in planning and computer games. ICAPS.
- Wu, C., Passenger, J., Song, Y. (2026). Tracing the Thoughts of a Coding Agent Playing ARC-AGI-3: Lessons for Continual Learning. arXiv:2610.11450.
- Wu, W. (2026). Kepler: Auditable World Models for ARC-AGI-3. arXiv:2610.00834.
- Zweiger, A. et al. (2025). Self-Adapting Language Models. arXiv:2506.10943.